An Uninterrupted Processing Technique-Based High-Throughput and Energy-Efficient Hardware Accelerator for Convolutional Neural Networks

Md Najrul Islam, *Graduate Student Member, IEEE*, Rahul Shrestha, *Senior Member, IEEE*, and Shubhajit Roy Chowdhury, *Senior Member, IEEE*

***Abstract*— This article proposes an uninterrupted processing technique for the convolutional neural network (CNN) accelerator. It primarily allows the CNN accelerator to simultaneously perform both processing element (PE) operation and data fetching that reduces its latency and enhances the achievable throughput. Corresponding to the suggested technique, this work also presents a low latency VLSI-architecture of the CNN accelerator using the new random access line-buffer (RALB)-based design of PE array. Subsequently, the proposed CNN-accelerator architecture has been further optimized by reusing the local data in PE array, incurring better energy conservation. Our CNN accelerator has been hardware implemented on Zynq-UltraScale+ MPSoC-ZCU102 FPGA board, and it operates at a maximum clock frequency of 340 MHz, consuming 4.11 W of total power. In addition, the suggested CNN accelerator with 864 PEs delivers a peak throughput of 587.52 GOPs and an adequate energy efficiency of 142.95 GOPs/W. Comparison of aforementioned implementation results with the literature has shown that our CNN accelerator delivers 33.42% higher throughput and 6.24× better energy efficiency than the state-of-the-art work. Eventually, the field-programmable gate array (FPGA) prototype of the proposed CNN accelerator has been functionally validated using the real-world test setup for the detection of object from input image, using the GoogLeNet neural network.**

***Index Terms*— Convolutional neural network (CNN), digital VLSI-architecture design, field-programmable gate array (FPGA), VGG-16 and GoogLeNet neural networks, VLSI.**

I. Introduction

CONVOLUTIONAL neural network (CNN) has steered the modern artificial-intelligence applications, such as audio, text, and visual object recognition, to achieve excellent accuracy [1], [2], [3], [4], [5], [6], [7]. Superior performance of CNN is calibrated based on the way of recognizing the text or object or audio-key-frame in the input image or audio data by searching for the presence of millions to several billions of trained feature parameters [7]. The CNN carries out such feature search by sweeping and performing arithmetic operations (such as multiplication and addition) with different filter weights that are associated with various features across the input data [8]. However, the superior recognition accuracy of CNN is achieved at the cost of extremely high computational complexity and massive data movements, incurring performance degradation and surge in energy consumption, respectively [7], [8]. Such adverse consequences refrain CNN from its efficient hardware implementation for battery operated devices, which are extensively used in contemporary edge applications, such as the Internet of Things, autonomous electric vehicle, and implanted biomedical devices [9]. Furthermore, such present-day applications demand higher computation throughput at lower power consumption [8]. Based on aforementioned discussion, substantial throughput and profound energy efficiency have become key requirements for the hardware implementation of CNN accelerators [10], [11]. On the other hand, when power hungry engine, such as graphic processing unit (GPU), is used as a core hardware for CNN processing [7], attempts are being made to accelerate CNN in an energy-efficient way [12]. Similarly, dedicated parallel-processing platforms, such as mobile GPU [13], tensor processing unit (TPU) [14], field-programmable gate arrays (FPGAs) [9], [12], [15], [16], application-specific integrated circuit (ASIC) [17], and in-memory computation (iMC) [18], are used to efficiently process the computations in CNN. Note that GPUs and TPUs consume hundreds of watts, which make them unfit for any edge applications. Subsequently, iMC and ASIC deliver highest energy efficiency; however, they have poor compatibility and scalability with the rapidly evolving CNN models due to the lack of reprogrammability. In contrast, the FPGA implementation of CNN accelerator offers reprogrammability, and if its hardware architecture is well designed, then it delivers adequate energy efficiency [8]. On the one side, GPU, TPU, ASIC, and iMC implementations cannot work as a stand-alone system for any application, as they are instead required to be interfaced with an external computer/controller. On the other side, contemporary FPGA boards with onboard computer and logic elements (such as Zynq-7000 SoC and Zynq Ultrascale+ MPSoC series of FPGA boards) can be implemented as complete stand-alone systems for various contemporary applications.

Energy efficiency of high-throughput parallel-computing CNN accelerator can be enhanced by extensively reusing the

Manuscript received 1 June 2022; revised 1 September 2022; accepted 21 September 2022. Date of publication 13 October 2022; date of current version 9 December 2022. This work was supported by the Indian Institute of Technology Mandi. *(Corresponding author: Rahul Shrestha.)*

The authors are with the School of Computing and Electrical Engineering, Indian Institute of Technology (IIT) Mandi, Mandi 175075, India (e-mail: d18064@students.iitmandi.ac.in; rahul_shrestha@iitmandi.ac.in; src@iitmandi.ac.in).

Color versions of one or more figures in this article are available at https://doi.org/10.1109/TVLSI.2022.3210963.

Digital Object Identifier 10.1109/TVLSI.2022.3210963

local data and minimizing the expensive read/write operations to and from the external memory. In addition, low precision computation can also be used to significantly increase the energy efficiency. In the reported work from [17], row-stationary data flow-based architecture of the CNN accelerator delivers promising energy efficiency by reducing the amount of off-chip memory access that requires a large number of shift registers to aid the local data reuse. However, such design is not a convenient choice for FPGA-based applications [8]. Nguyen et al. [19] could achieve higher energy efficiency using lower precision at the cost of degradation in accuracy. Later, Nguyen et al. [20] have improved such accuracy using the layer-specific optimization for mixed data flow, using the mixed precision. Similarly, [9] presents software-based reconfigurability using the dual-loop arithmetic module at the cost of throughput and energy efficiency, due to un-optimized data movement. In [8], the energy efficiency has been enhanced by reducing the off-chip memory access with the aid of the kernel partitioning technique. However, it requires extra pre- and postprocessings on the input and the output data. Furthermore, despite the presence of parallel computing, real-world yield in computational throughput of most CNN accelerator falls below the theoretically estimated throughput due to under utilization of available resources and underlying problem of interrupted data supply [7]. Recent contribution from [21] showed that, in the worst case scenarios, a state-of-the-art complex model, such as GoogleNet, utilizes only 6.6% of processing elements (PEs) in an accelerator. Therefore, to circumvent the aforementioned issues and challenges, this article presents an energy as well as hardware efficient, high throughput, and reconfigurable FPGA-based CNN accelerator for object recognition application. Our main contributions in this article are as follows.

1) A new uninterrupted processing technique has been proposed to reduce the latency of CNN accelerator that enhances its throughput and energy efficiency.
2) In addition, we have suggested a new data-reusability process to achieve maximum reuse of local data without data shifting in CNN accelerator to suppress the data movement that eventually reduces its energy consumption.
3) Presented a new random access line-buffer (RALB)-based hardware architecture of CNN accelerator to realize the above-proposed techniques.
4) The proposed CNN accelerator has been hardware implemented in various FPGA platforms, and their results are compared with the state-of-the-art implementations.
5) Eventually, the hardware prototype of our CNN accelerator in Zynq-UltraScale+ MPSoC-ZCU102 FPGA board has been functionally validated in real-world test setup that classifies object from an input image with the aid of the GoogLeNet CNN model.

This article has been organized as follows. In Section II, an overall system-level view of our work has been presented, along with brief mathematical background. It also comprehensively discusses the research challenges that are addressed in this article. Furthermore, Section III presents the proposed technique and new VLSI architectures for the CNN accelerator. Following that, the implementation results, discussion, comparison, and validation are included in Section IV. Eventually, this article concludes in Section V.

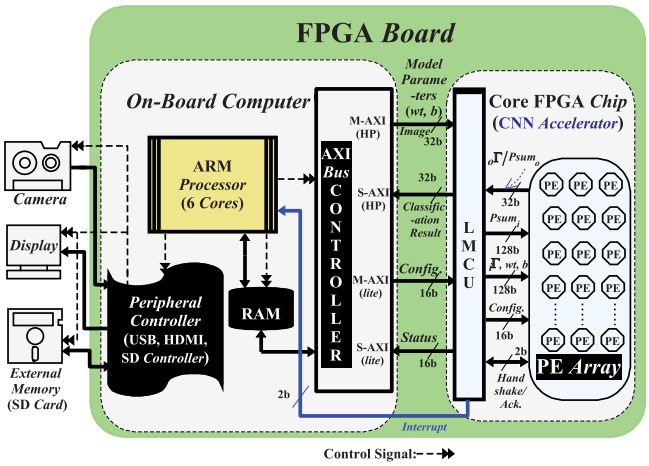

Fig. 1. Schematic representation of the hardware level system using FPGA board and other supporting peripherals.

II. System Model, Mathematical Prerequisite, and Research Challenges

A. System Model

An overview of the hardware level system for an object detection application has been schematically shown in Fig. 1. It comprises of four major hardware components: 1) camera; 2) FPGA board (Zynq MPSoC evaluation board); 3) external memory (secure digital (SD) card); and 4) display. Here, the FPGA board consists of onboard computer that can run software programs, written in high-level language. Specifically, it has an advanced RISC machine (ARM) processor with six cores, DDR4 random access-memory (RAM), and various peripheral controllers, such as universal serial bus (USB) controller (that interfaces camera with the FPGA board), external SD-card controller, and display controller, as shown in Fig. 1. It also shows that the CNN accelerator is implemented using configurable logic blocks, multipliers, and embedded block RAMs of core FPGA chip on the board. Furthermore, the SD card contains the pretrained model parameters and additional software programs. As illustrated in Fig. 1, the CNN accelerator has been designed using a local memory and control unit (LMCU) and an array of PEs. Each of these PEs comprises of a multiply-and-accumulate (MAC) unit and a comparator unit. Here, the LMCU manages the operations of PE array and accordingly moderates the data flow in our CNN accelerator. Its operation is iteratively carried out via closed loop data flow between the LMCU and the PE array. In every iteration, the PE array receives the input feature matrix (${}_i\Gamma$) and other parameters, such as filter weights (wt) as well as biases (b) from LMCU. Subsequently, the PE array produces the output feature matrix (${}_o\Gamma$) that is passed back to LMCU. In such iterative process, ${}_o\Gamma$ from one operation is reused as ${}_i\Gamma$ for the computation in the subsequent layer. This process continues until the operations for all the layers are

accomplished, as discussed further in Section II-B. As shown in Fig. 1, the CNN accelerator has been interfaced with onboard computer via high-performance Advanced eXtensible Interface (AXI) buses. Here, ${}_i\Gamma$ for the first layer of CNN is the input image, and ${}_o\Gamma$ for the last layer of CNN is the computed probability of each class in the model. To begin with object detection process, when the system initially boots up, the onboard computer of FPGA transfers the model information along with trained wt and b parameters to configure the CNN accelerator. Thereafter, once the accelerator is fully configured with the model specification, onboard computer begins the processing of input images from external source (i.e., camera) and starts feeding/off-loading them to the CNN accelerator. Subsequently, it performs a number of operations on the images for all convolution (CONV) and fully connected (FC) layers to search for different class features. Thus, it classifies the object that is present in the input image and computes its probability.

B. Mathematical Prerequisite for CNN-Accelerator Computations

Various operations performed for the CONV layers are classified as MAC, pooling, and activation operations. Search for the presence of certain feature determined by trained filter kernel/matrix in ${}_i\Gamma$ is performed by the strided sliding window-based MAC operation. It is mathematically expressed as follows:

$$ {}^{C}_{o}\Gamma^{l}_{x,y,z} = \sum_{p=1}^{\check{X}} \sum_{q=1}^{\check{X}} \sum_{r=1}^{\hat{D}} \left(W^{l}_{p,q,r,z} \times {}_i\Gamma^{l}_{a,b,r} \right) + b^{l}_{z} \tag{1} $$

where ${}^{C}_{o}\Gamma^{l}$ represents the 3-D matrix of detection score for all the features searched by all filters in the lth CONV layer. Here, $a = s \times x + p$, $b = s \times y + q$, and $W^{l}_{p,q,r,z}$ is a 4-D filter weight for the lth layer. In addition, $\check{X}$ is the width/height of the filter kernel, $\hat{D}$ is the depth of ${}_i\Gamma^{l}$, and s is the size of stride in that layer. On the other hand, max pooling is carried out on ${}^{C}_{o}\Gamma^{l}$ to maximize the prominent scores, and this operation can be expressed as follows:

$$ {}^{P}_{o}\Gamma^{l}_{x,y,z} = \max\left\{ {}^{C}_{o}\Gamma^{l}(x : \alpha, \;\; y : \beta, \;\; z) \right\} \tag{2} $$

where $\alpha = x + p_w$, $\beta = y + p_h$, and $\max\{{}^{C}_{o}\Gamma^{l}(x : \alpha, \;\; y : \beta, \;\; z)\}$ denotes the extraction of largest element from the $p_w \times p_h$ matrix window with an origin at (x, y) inside ${}^{C}_{o}\Gamma^{l}_{z}$. Rectified linear unit (ReLU) activation is applied on ${}^{P}_{o}\Gamma^{l}$ to discard all the negative feature scores and is given by the following:

$$ {}^{R}_{o}\Gamma^{l}_{x,y,z} = \max\left\{0, \; {}^{P}_{o}FM^{l}_{x,y,z}\right\}. \tag{3} $$

This ${}^{R}_{o}\Gamma^{l}_{x,y,z}$ is used as ${}_i\Gamma^{l+1}$ in the subsequent $(l+1)$th CONV layer. For FC layers, each element of ${}_o\Gamma$ is the weighted sum of all elements of ${}_i\Gamma$ and their corresponding biases. This operation is mathematically expressed as follows:

$$ {}^{M}_{o}\Gamma^{F_i}_{k} = \sum_{n=1}^{N} {}_i\Gamma^{F_i}_{k} \times W^{F_i}_{n,k} + b^{F_i}_{k}. \tag{4} $$

Subsequently, soft max activation is applied on ${}^{M}_{o}\Gamma^{F_i}_{k}$ to compute the probability of each class in the model that classifies object in the image. This is given by the following:

$$ p_n = \left(\frac{e^{\left({}^{M}_{o}\Gamma^{F_i}_{n}\right)}}{\sum_{k=1}^{N} e^{\left({}^{M}_{o}\Gamma^{F_i}_{k}\right)}} \right) \tag{5} $$

where p_n is the probability score of the nth class among the total of N classes in the model. Finally, the CNN accelerator returns the prediction result to the onboard computer, which uses it to display on the monitor.

C. Research Challenges

1) Energy-Efficient Data Flow: The number of operations required by (1)–(5), in their corresponding hardware architectures, varies over a wide range that proliferates the computational complexity. From an implementation aspect, the CNN accelerator must process such computationally complex operations in high-speed and energy-efficient manners by mapping them over a large array of PEs. Thus, it exploits high degree of parallelism and extensive reusability of local data within the periphery of PE array to achieve higher computation throughput and lower energy consumption, respectively. Both these performance factors are primarily dependent on the availability of data locally within the array of PEs. Chen et al. [17] have achieved excellent energy efficiency from their implemented architecture with the aid of row stationary approach. Here, wt, ${}_i\Gamma$, and *Psums* are reused in horizontal, diagonal, and vertical directions, respectively, in the PE array of the CNN accelerator. It showed that the row stationary approach achieves higher energy efficiency in comparison with the weight or output stationary [22], [23] approach. However, the row stationary data flow requires continuous shifting of data over a large number of shift registers in all directions. The work reported in [8] demonstrated that the design with such a large number of shift registers is infeasible to be implemented on the FPGA platform. Thereby, [8] proposed a kernel partition technique that downscaled the number of memory accesses to achieve higher energy efficiency in the FPGA-based reconfigurable architecture, at the cost of highly complex pre- and post-processing. Furthermore, [20] showed that by proper optimization of data precision over different layers, all weights of CNN model can be stored in the block RAM (BRAM) of FPGA to avoid off-chip DRAM access for an excellent energy efficiency. However, it requires to redesign the hardware architectures from extremely low level for each of the different CNN models, and it must retrain the model to preserve accuracy. Therefore, our work presents RALB-based modified row-stationary data flow where both ${}_i\Gamma$ and wt values are maximally reused without shifting them from one PE to another PE that alleviates the latency as well as conserves the energy. Hence, detailed working of the RALB-based data flow has been clearly discussed in Section III-B2.

2) Throughput Alleviation Due to Interruption: To directly mapping (1)–(5) on hardware, it requires enormous size of PE array in CNN accelerator. However, such mapping is not feasible due to various design-constraint limitations, such as memory bandwidth, area, and power budgets. Therefore, computations in the CNN-accelerator hardware take place in a phased manner over a number of processing

iterations [7], [17]. During each iteration, the PE array of CNN accelerator computes partial sum (*Psum*), which is a portion of the output feature map that is later combined with other partial sums to generate final output feature map ${}_o\Gamma$ for (1)–(5) based on the configuration information from LMCU, as shown in Fig. 1. In the energy-efficient data flow-based conventional CNN accelerator, such as [17], [22], and [23], between every two consecutive iterations when the data are being fetched from the LMCU to the local storage inside each PE, the PE array cannot operate and remains idle that consequents in frequent throttling. As a result, it increases the latency and eventually degrades the achievable throughput of the CNN accelerator. There have been various attempts to reduce the latency; for example, [24] uses the pin-pong approach to reduce the latency. However, it does not support local reuse of data, incurring higher memory bandwidth; as a result, it is unable to completely minimize the idle time for smaller sized filters due to bandwidth limitations. Furthermore, [25] combined a fine-grained column-based pipeline approach with ping-pong architecture based-filter storage to reduce the latency for filter loading; however, it does not minimize the latency incurred for loading ${}_i\Gamma$, and it does not support reuse of local data. Moreover, the ping-pong approach demands double the amount of required memory (for example, $2 \times k_i$ kernel storage for k_i kernel groups in [25] and two buffers for wt, ${}_i\Gamma$, and Psums/${}_o\Gamma$ in [24]). Hence, this work proposes a new technique and a hardware architecture for an energy-efficient CNN accelerator that mitigate such frequent throttling issue of PE array by efficiently optimizing the way the data are fetched from LMCU and stored within the periphery of PE array, thus enhancing throughput and energy efficiency of the proposed CNN accelerator.

III. Proposed Technique and Architecture

A. Proposed Uninterrupted-Processing Technique

As discussed in Section II-C, if t_{f_i} represents the data fetching time between LMCU and local storage of PE in the ith iteration, then Fig. 2(a) and (b) shows the timing diagrams for the conventional and proposed CNN accelerators, respectively. Here, t_{c_i} denotes data computation time of PE array in the ith iteration, where each of these iterations consumes the time duration of t_{itr_i}. In the ith iteration of the conventional CNN accelerator, a process (denoted by P_i) in the PE array that computes *Psums* begins only after completing the data fetching event (F_i) from LMCU to PE array, as shown in Fig. 2(a). During this t_{f_i} time of F_i, the PE array remains idle. Thus, effective duration of every ith iteration is $t_{\text{itr}_i} = t_{f_i} + t_{c_i}$. On the other hand, the proposed uninterrupted-processing technique mitigates this t_{f_i} time and, hence, alleviates the latency that consequently enhances the throughput of our CNN accelerator. Various steps of this technique have been hierarchically presented in Algorithm 1, and its primary notion is schematically illustrated in Fig. 2(b). Its key idea is to concurrently perform both F_i and P_i, rather than processing them in a sequential manner. To elaborate further, the computation of P_i can start once the minimum number of data, say r, has been fetched from LMCU to PE array (referring lines 5–11 in Algorithm 1) that requires a prefetch duration of t_{pf_i}, as shown in Fig. 2(b). This t_{pf_i} is a part of total fetch duration t_{f_i} of the ith iteration, and note that $t_{pf_i} \ll (t_{f_i}/2)$. Subsequently, remaining data are fetched during the computation of the ith iteration, referring lines 12–15 in Algorithm 1.

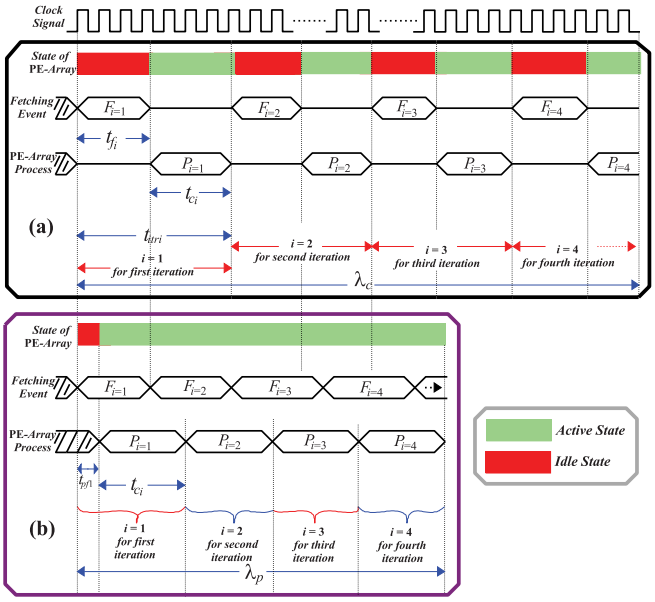

Fig. 2. Timing diagrams for (a) conventional and (b) proposed techniques where λ_c and λ_p are their latencies, respectively.

Algorithm 1 Proposed Uninterrupted-Processing Technique for the CNN Accelerator

1: Determine n and r; ▷ n denotes number of required iterations and r denotes the minimum number of data required to begin P_i process.
2: **Initialization:** i=0 and j=0; ▷ i and j count the number of processed iteration and the number of data that has been fetched, respectively.
3: Begin $F_{i=1}$; ▷ Start pre-fetching of data for $itr_{i=1}$.
4: **for** $i \leq n$ **do**
5: **if** $j \leq r$ **then** ▷ Enough data has not been fetched yet.
6: Continue $F_{i=1}$; ▷ Fetching continues for $itr_{i=1}$.
7: j = j+1; ▷ Count the number of data pre-fetched for $itr_{i=1}$.
8: Return to Step 5; ▷ Fetching continues for $itr_{i=1}$.
9: **else** ▷ Enough Data Fetched to Begin Computation.
10: i = i+1;
11: Begin P_i; ▷ Computation begins for itr_i.
12: **if** F_i is Complete **then**
13: Begin F_{i+1}; ▷ Begin pre-fetching data for itr_{i+1}.
14: **else**
15: Continue F_i;
16: **end if**
17: **if** P_i is Complete **then**
18: Return to Step 10. ▷ Begin computation for for itr_{i+1}.
19: **else**
20: Continue P_i.
21: **end if**
22: **end if**
23: **end for**

Here, t_{f_i} is shorter duration than t_{c_i}, and thereby, F_i finishes much earlier than the end of operation P_i. Thereby, we propose to start the prefetch for next $(i+1)$th iteration before the P_i operation ends, referring line 13 in Algorithm 1 and schematically represented in Fig. 2(b). Since the minimum r data have been already prefetched for the $(i+1)$th iteration during P_i of the ith iteration, P_{i+1} commences instantaneously at the end of P_i. Such phenomenon effectively mitigates the idle time of PE array during F_{i+1} (i.e., t_{f_i} is mitigated,

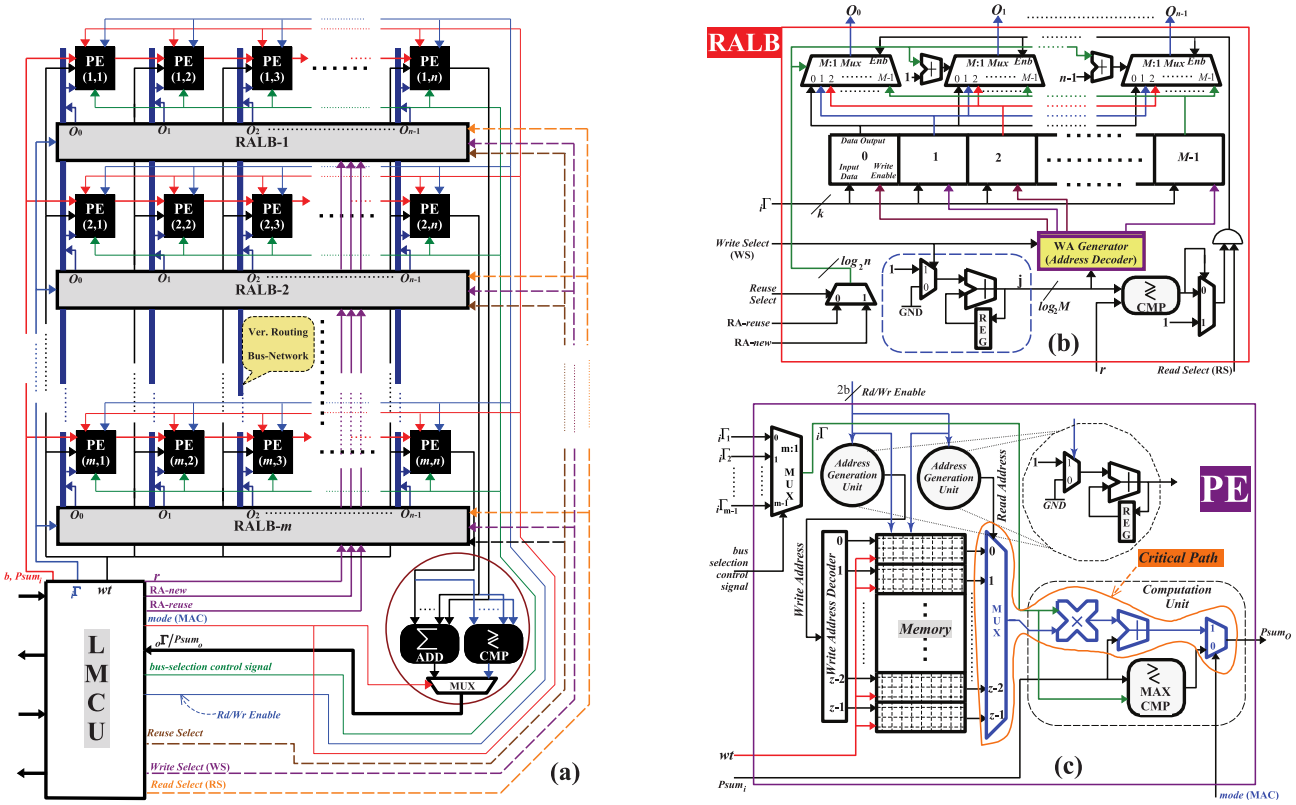

Fig. 3. (a) Proposed VLSI architecture of the CNN accelerator. (b) Suggested internal microarchitecture of RALB. (c) PE microarchitecture.

as shown in Fig. 2). As a result, the effective duration for each iteration becomes approximately equal to computation time required by P_i (i.e., $t_{\text{itr}_i} \approx t_{c_i}$). Hence, for the CNN model that requires n iterations, the effective latency reduces from $\lambda_c = \sum_{i=1}^{n}(t_{f_i} + t_{c_i})$ to $\lambda_p = (\sum_{i=1}^{n} t_{c_i}) + t_{pf1}$, where λ_c and λ_p represent the latencies of conventional and proposed CNN accelerators, respectively, thus conserving the time duration of $\approx \sum_{i=1}^{n}(t_{f_i})$.

B. Proposed VLSI Architectures

1) Low-Latency CNN-Accelerator Architecture: Referring to Fig. 2(b), the PE array of CNN accelerator must simultaneously perform two operations: 1) store the new data fetched from LMCU and 2) supply data to the computation unit of PEs. Moreover, the proposed architecture for CNN accelerator needs to achieve *reuse of local data* without using large number of shift registers to conserve both energy and hardware resources. To incorporate these features in the suggested CNN accelerator, this work presents new architecture for locally storing and reusing the data within the periphery of PE array. Hence, the proposed generic architecture of the CNN accelerator that supports $m \times n$-sized PE array is shown in Fig. 3(a). In conventional CNN accelerators, such as [16] and [17], both filter weights and ${}_i\Gamma$ values are stored in each of the PEs using the single-port SRAM, due to which the PE of such conventional design is incapable of operating when the data are being fetched. Unlike, our work changes the wt storage inside the PE to dual port memory to achieve simultaneous read as well as write (for wt prefetch) feature and changes the ${}_i\Gamma$ storage by introducing the RALB-based approach to achieve simultaneous read–write feature along with *local data reuse* without shifting ${}_i\Gamma$ from one PE to another PE. Schematic representation of the RALB-based approach in the proposed CNN accelerator is depicted in Fig. 3(a). It shows that each RALB is connected to a single or multiple row(s) of PEs, at any instance of time, to store their ${}_i\Gamma$ values. Furthermore, the suggested microarchitecture of RALB is presented in Fig. 3(b) that incorporates steering logics and an $M \times k$-sized memory. This RALB is capable of writing k bit of data to any of these M memory locations pointed by j write address (WA) that is steered via the WA generator (i.e., address decoder), as illustrated in Fig. 3(b). Once the value of j WA exceeds the magnitude of r (i.e., minimum number of data), the RALB provides n number of k bit data to the computation units of n PEs, corresponding to the row with which RALB is connected to begin the P_i process, referring line 11 from Algorithm 1. Furthermore, RALB continues to store new data from outside the PE array, while simultaneously providing the data to n PEs. Thus, the aforementioned process enables the proposed architecture to concurrently perform the operations of both F_i and P_i. In addition, it avoids the possible problem of read-before-write due to the fact that read rate (i.e., frequency at which read address increases) is $z\times$ smaller than write rate (rate at which WA increases). The reason being, each element of ${}_i\Gamma$ fed to PE is reused $z\times$ for MAC operation with z filter weights that are stored inside the PE, as shown in the microarchitecture of each PE from Fig. 3(c).

Since RALB stores $M/\hat{X}$ rows of ${}_i\Gamma$ values and $\hat{X}+r$ is less than M, RALB can prefetch $M-\hat{X}$ number of data for the next iterations (i.e., F_{i+1} begins), referring line 13 in Algorithm 1. Note that $\hat{X}$ denotes width of each row of ${}_i\Gamma$. As the data are prefetched, P_{i+1} computation for the next iteration can start once P_i ends, under the condition that the filter weights are prefetched and stored in the filter storage of PE, as presented in Fig. 3(c). It consists of a small $z \times k$ sized memory for storing z number of filter weights and a computation unit that can be configured for MAC or max operation. With the aid of bus-selection control signal to PE, its computation unit is routed with ${}_i\Gamma$ values from the respective RALB, which is connected with the PE, as shown in Fig. 3(a) and (c). For performing MAC and pooling operations in PE, partial sum is fed as Psum_i to the PE architecture. When the *mode*(MAC) signal is asserted high, the MAC module is activated that computes Psum_o, as shown in Fig. 3(c). Otherwise, if *mode*(MAC) signal is asserted low, then the max comparator (i.e., MAX CMP) is activated that allows Psum_o to be assigned with the maximum value between ${}_i\Gamma$ and Psum_i (i.e., max operation for pooling or ReLU task). In our accelerator, ReLU operation can be achieved in two ways: 1) passing null values to the Psum_i port and configuring the PE for max operation or 2) achieving pre-MAC ReLU by deactivating the multiplier during the MAC operation to produce a zero value of output in the presence of negative sign in ${}_i\Gamma$. Later approach reduces the requirement of data movement by a factor of 1/3, which enhances the energy efficiency. It can be combined with clock gating to further reduce the energy consumption. Since both W and ${}_i\Gamma$ are prefetched, the P_{i+1} process instantaneously commences after the P_i process ends and, hence, incurring the time duration for an iteration to be $t_{\text{itr}_i} \approx t_{c_i}$, referring Fig. 2(b).

2) Data Reusability in PE Array Architecture: Using vertical routing bus network in PE array from Fig. 3(a) and bus-selection control signal for PEs in Fig. 3(c), any row of PEs in the PE array can be connected to any RALB within its periphery. This allows our CNN-accelerator architecture to exploit the local reuse of ${}_i\Gamma$ values throughout the period of horizontal strides in a row. Such ${}_i\Gamma$ values stored in an RALB remain stationary, while the LMCU increases the read address of RALB by s (note that s denotes the stride size, as discussed earlier in Section II-A), from where the data are fed to PEs, thus horizontally reusing the ${}_i\Gamma$ values without moving them from one PE to another. For vertical reuse of ${}_i\Gamma$ values, let us assume that the xth RALB is connected to the ith row of PE array, and the $(x+1)$th RALB is connected to the $(i+1)$th row of PE array, and the remaining rows of PE array are connected in the similar fashion. Now, during each vertical strides, x index increments by s and, hence, $(x+s)$th RALB connect to the ith row of PE array, and the same pattern is followed by rest of the rows in PE array. Since the number of RALBs is limited, x periodically resets to zero after it exceeds the maximum RALB count, and such cycle continues till the end of all iterations. Note that during each of the vertical strides, LMCU resets the read addresses of RALBs (whose data are to be reused by another row of PE array) to the new address where it was at the beginning of the previous vertical stride. Furthermore, LMCU increases the read addresses of those RALBs, which contain the new lines of ${}_i\Gamma$ for current vertical stride. Therefore, each ${}_i\Gamma$ value is reused $z(\check{X}-s)^2$ times, and hence, if z is large enough to fit all the values of the filter weights in a layer, then each ${}_i\Gamma$ value needs to be fetched only once from the LMCU. Moreover, the filter weights stored in local storage inside PE remain stationary throughout the period of all horizontal and vertical strides. Thus, each of these filter weights has been reused for $(\hat{X}-s)^2$ times. Thereby, aforementioned process enables the proposed CNN accelerator to conserve its energy and latency.

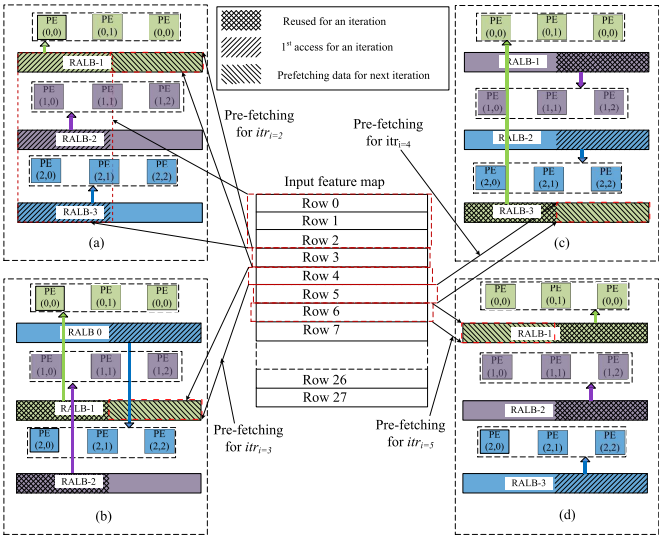

Fig. 4. Schematic representation of the suggested data-reusability process for 3×3 PE array in the CNN accelerator. Processing during (a) $\text{itr}_{i=1}$, (b) $\text{itr}_{i=2}$, (c) $\text{itr}_{i=3}$, and (d) $\text{itr}_{i=4}$.

For example, assume 3×3 PE array that comprises of three RALBs, as shown in Fig. 4. Here, $s = 1$, and the same colored PE rows and RALB are connected with each other, such that single RALB stores up to two rows of 28×28 input feature map ${}_i\Gamma$. Consider each vertical stride as an iteration (itr_i). Fig. 4(a) shows that initially at $\text{itr}_{i=1}$ iteration, RALB-0, RALB-1, and RALB-2 are connected with the first, second, and third rows of PE array, respectively. Hence, they sequentially store Rows 0–3 of ${}_i\Gamma$ at their respective locations between the addresses 0–27. During the computation in $\text{itr}_{i=1}$, each PE row gets data from these addresses 0–27 of their respective connected RALBs. For each horizontal stride, the read addresses of these RALBs increase by one because $s = 1$. Furthermore, Row 3 of ${}_i\Gamma$ is prefetched at the locations between addresses 28–55 (i.e., second half) of RALB-0. After the first vertical stride for $\text{itr}_{i=2}$ iteration, the first row of PE array connects with RALB-1 to reuse the data of Row 1 of ${}_i\Gamma$, the second row of PE array connects with RALB-2 to reuse the data of Row 2 of ${}_i\Gamma$, and the third row of PE array connects with RALB-0 to use the data of Row 3 of ${}_i\Gamma$ for the first time, as shown in Fig. 4(b). Thus, during horizontal strides, the read addresses of RALB-1 and RALB-2 that contain the reused data vary between 0 and 27, and the read addresses of RALB-0 that contain new data vary between 28 and 55. Similar approach is followed in both $\text{itr}_{i=3}$ and $\text{itr}_{i=4}$ iterations, as illustrated by

TABLE I
CONTRIBUTION OF DIFFERENT FILTERS IN TOTAL COMPUTATION OF VARIOUS CNN MODELS

	$\mathcal{C}_{xy}$ Values of Filters for Different CNN Models				
CNN Model	1×1	3×3	5×5	7×7	11×11
AlexNet	-	42%	47%	-	11%
VGG-16	-	100%	-	-	-
GoogLeNet	22%	63%	8%	7%	-
MobileNet	10%	90%	-	-	-

Fig. 4(c) and (d), respectively. For $\text{itr}_{i=3}$, $x+s$ is greater than two, and hence, $x+s$ resets to zero value. Again, RALB-0 gets connected with the first row of PE array, and the same process continues for RALB-1 and RALB-2.

IV. IMPLEMENTATION RESULTS, COMPARISONS, AND HARDWARE VALIDATION

A. FPGA Implementation Results

In the proposed architecture of $m \times n$ PE array, m determines three imperative factors: 1) length of vertical routing network; 2) size of bus selection multiplexer in PE; and 3) data movement latency between single RALB and PE that belongs to the farthest PE row (i.e., RALB-0 to the mth PE row). Similarly, n determines the width of vertical routing network. The computation throughput is $\Theta_T \propto N_{\text{PE}} \times V$, where N_{PE} is the number of PEs in the CNN accelerator (i.e., $N_{\text{PE}} = m \times n$), and V represents the PE efficiency, which is given by $V = N_{\text{PE-util.}}/N_{\text{PE}}$, such that $N_{PE-util.}$ denotes the number of PEs utilized. To achieve higher PE efficiency that maximizes the Θ_T value, both m and n must be compatible with the filter shape of the CNN model. A summary of computation shares contributed by different filters in five most popular CNN models has been presented in Table I. Here, the computation share contributed by each filter of size $x \times y$ is calculated as $\mathcal{C}_{xy} = (\mathcal{M}_{xy}/\mathcal{M}_{\text{tot}}) \times 100\%$, where $\mathcal{M}_{xy}$ and $\mathcal{M}_{\text{tot}}$ represent the number of MACs contributed by single filter (of size $x \times y$) and all the filters, respectively. As presented in Table I, majority of computations in the state-of-the-art CNN models are dominated by smaller filters, mostly 3×3-sized filter, rather than large filters with the size, such as 7×7 or 11×11.

Since a CNN accelerator cannot be 100% efficient for all the filter shapes, m must be chosen in a way that it emphasizes only those filters, which contribute highest computations, along with the abovementioned hardware aspects. Hence, the proposed CNN accelerator consists of a dynamically configurable 36×24-sized PE array that is segregated into 16 smaller clusters to keep the effective value of m and n smaller. Each of these clusters comprises of a 9×6-sized PE array along with nine RALBs. Every single RALB can store 1024 pixels of ${}_i\Gamma$, and thus, effective values are $m = 9$ and $n = 6$ for these clusters. They are individually controlled by the LMCU, and each of them can simultaneously perform convolution operations for six 3×3-sized filters that delivers 100% of PE efficiency ($V = 1$) or two 5×5-sized filters with 92% of PE efficiency ($V = 0.92$) or 54 1×1-sized filters with 100% PE efficiency ($V = 1$) or only one 7×7-sized filter with 91% PE efficiency ($V = 0.91$). Therefore, the proposed VLSI-architecture of the RALB-based CNN accelerator has been hardware implemented on FPGA evaluation board (Xilinx Zynq-UltraScale+ MPSoC-ZCU102 board). Furthermore, gate-level synthesis and static-timing analysis of suggested CNN accelerator indicate that its critical path lies in the PE microarchitecture, and hence, the longest-path delay includes two multiplexers, one adder and one multiplier delays, as shown in Fig. 3(c). Thus, the proposed CNN accelerator is capable of operating at a maximum clock frequency of 340 MHz, when implemented on the Zynq-UltraScale+ FPGA board. Eventually, FPGA hardware resources consumed by the aforementioned implementation have been presented in Table II.

B. Comparison With the State-of-the-Art Implementations

For fair comparison with other reported works in Table II, we have also synthesized and post-route simulated the proposed CNN accelerator in the same FPGA platforms used by these implementations from the literature. Due to slower BRAMs of Xilinx Virtex-7 (VC709 and VC707) and Xilinx ZYNQ-7000 (ZC706) boards, the proposed CNN accelerator when implemented on these FPGA platforms could operate at a maximum clock frequency of 200 MHz. As discussed above, Θ_T is an imperative performance metric that indicates the number of computations performed per second by the CNN accelerator. It is computed as $\Theta_T = (2 \cdot N_{\text{PE}} \cdot f_{\text{clk}} \cdot U \cdot V)$ GOPs, where f_{clk} denotes an operating clock frequency of the CNN accelerator, GOPs refer to giga operations per second, and U is the activity rate of PEs that is given by $U = t_{\text{ci}}/t_{\text{itr}_i}$. Since each PE performs two operations: 1) multiplication and 2) addition in each clock cycle, referring Fig. 3(c), we have incorporated a factor of 2 in the aforementioned Θ_T expression. With the aid of the proposed interrupt-free processing, $t_{\text{itr}_i} \approx t_{c_i}$ in our CNN accelerator and, hence, the value of $U = 1$.

Furthermore, energy efficiency is expressed as $\epsilon = \Theta_T/\mathcal{W}$, where $\mathcal{W}$ represents the average power consumption that is computed as $\mathcal{W} = (\varepsilon_c + \psi_m)/t$. Here, ε_c is the computation cost for each data, which depends on the design logic and its precision. Similarly, ψ_m is the energy required to move the data from (or to) storage location, at different hierarchy, to (or from) the computation unit of PE. In our architecture, ψ_m can be expressed as $\psi_m = a \cdot \alpha + b \cdot \beta + c \cdot \gamma$, where a is the number of times the data have been loaded from DRAM to LMCU, and α is the energy consumption associated with each of these data movement. Similarly, b and c are the number of times the same data have been loaded from LMCU-to-RALB and RALB-to-PE, respectively, and their respective energy costs are given by β and γ. Note that b and c are inversely proportional to a and b, respectively. Among α, β, and γ, the smallest value is γ, whereas $\alpha \approx 200 \times \gamma$, and $\beta = 6 \times \gamma$ [17]. Since the magnitude of ψ_m is higher than ε_c, it is necessary to minimize the costly data movements, which reduces a and b values that can be compensated by increasing the c value. In the proposed CNN accelerator, each ${}_i\Gamma$ value is reused $z(\check{X}-s)^2$ times, and each of the filter weight is reused $(\hat{X}-s)^2$ times, as discussed earlier in Section III-B2. Thereby, for the ${}_i\Gamma$ value, b is $z(\check{X}-s)^2$ times smaller than c, and a is (number of filters)/z times smaller than b. Similarly,

TABLE II
FPGA AND ASIC IMPLEMENTATION-RESULTS COMPARISON OF THE PROPOSED CNN ACCELERATOR WITH RELEVANT STATE-OF-THE-ART WORKS

	[17]	[12]	[16]	[9]	[8]	[8]	[20]	**Proposed**				
Platform	65 nm	ZC706	ZC706	VC707	VC709	VC709	VC707	**ZC706**	**VC707**	**VC709**	**ZCU102**	
Clk. Freq. (MHz)	200	200	150	200	200	200	200	**200**	**200**	**200**	**340**	**340**
LUTS (in kilo)	NA	–	182.616	–	121.472	121.472	280.4	**189.591**	**102.936**	**102.936**	**103.985**	**103.985**
FFs (in kilo)	NA	–	127.653	–	159.872	159.872	220.6	**12.721**	**20.474**	**20.474**	**20.631**	**20.631**
BRAMs (36kb)	NA	–	486	–	467	467	715.5	**480**	**640**	**640**	**640**	**640**
N_{PE}	168	576	780	64	664	664	2515	**864**	**864**	**864**	**864**	**864**
Precision (in bits)	16	16/8	16	16	16	16	Mixed(16-1)	**16**	**16**	**16**	**16**	**16**
CNN Model	AlexNet	Mod. AlexNet	VGG-16	VGG-16	AlexNet	VGG-16	RESNET152	**VGG-16**	**VGG-16**	**VGG-16**	**VGG-16**	**GoogLeNet**
Θ_T (GOPs)	46.1	198.1	187.8	12.5	220	230.1	726	**345.6**	**345.6**	**345.6**	**587.52**	**576.7**
η_{PE} (MOPs/PE)	274	343	241	195	331	347	288.7	**400**	**400**	**400**	**680**	**667**
Power Consumption (W)	0.278	NA	NA	NA	9.61	9.61	–	**1.89**	**1.73**	**1.78**	**4.11**	**4.11**
Energy Effn. (GOPs/W)	166.2	NA	14.22	NA	22.9	22.9	–	**182.85**	**199.77**	**194.1573**	**142.95**	**140.31**

for filter weights, both a and b are $(\hat{X} - s)^2$ times smaller than c. Hence, the proposed RALB-based CNN accelerator consumes an average total power of 4.109 W while operating at a peak clock frequency of 340 MHz, when implemented on the Zynq-UltraScale+ FPGA board. This total power consumption of 4.109 W is composed of the dynamic and static powers of 3.383 W (i.e., 82%) and 0.726 W (i.e., 18%), respectively. Such power consumption of the proposed design is 2.34× lesser than the state-of-the-art FPGA implementation results from [8]. Based on this total power consumption, the energy efficiency (i.e., $\epsilon = \Theta_T/\mathcal{W}$) of our architecture is 140.95 GOPs/W, which is 6.24× higher than the contemporary implementation from [8].

To demonstrate the reconfigurability of our design, it has been configured and implemented for both VGG-16 and GoogLeNet neural networks, as their implementation results are presented in Table II. Using the proposed uninterrupted processing technique along with the RALB-based PE array mapping, while processing VGG-16 and GoogLeNet CNN models, 100% (i.e., $V = 1$) and 98.16% (i.e., $V = 0.9816$) of PEs, respectively, have been utilized by the proposed CNN-accelerator architecture. Therefore, based on aforementioned quantifications, our design delivers the throughput of 576.7 GOPs at a frame rate of 202 frames/s and 587.52 GOPs at a frame rate of 18.9 frames/s while processing GoogLeNet and VGG-16 neural networks, respectively, as presented in Table II. This work presents a figure-of-merit (FOM), which is termed the throughput density (η_{PE}), that is computed as $\eta_{\text{PE}} = \Theta_T/N_{\text{PE}}$. This FOM indicates the number of operations performed in 1 s per PE, and hence, its higher value is desirable. Among all the relevant reported works [8], [9], [12], [16], [17] in Table II, the state-of-the-art implementation (using Xilinx Virtex-7 VC709 FPGA board) of [8] delivers the highest throughput of $\Theta_T = 230.1$ GOPs and the PE efficiency of $\eta_{\text{PE}} = 347$ MOPs/PE. Moreover, with the aid of our uninterrupted processing technique, the proposed CNN-accelerator architecture when implemented in the same FPGA platform delivers a throughput of 345.6 GOPs, which is 33.42% better than the one reported by [8]. Furthermore, the suggested architecture achieves the PE efficiency of 400 MOPs/PE, which is 13.25% better than η_{PE} of [8], as shown in Table II. It also shows that the proposed CNN accelerator, when implemented on the Xilinx Zynq-UltraScale+ ZCU102 FPGA board, delivers the highest throughput and the PE efficiency of 587.52 GOPs and 680 MOPs/PE, respectively. In terms of hardware utilization, our design requires moderate consumption that is comparable to the reported implementations, as presented in Table II. On the other side, the proposed CNN accelerator is a scalable design, as its Θ_T increases with the size of PE array by maintaining nearly constant V and η_{PE} values. To demonstrate the same, Fig. 5 presents the variations of Θ_T, V, and η_{PE} with the increasing sizes of PE array from 7×7 to 36×48 for both VGG-16 and GoogLeNet CNN models.

C. Peak Throughput Issues Due to DRAM Bandwidth Limitation of FPGA

Majority of the CNN accelerators, such as [24], [26], and [27], face difficulties in achieving their peak throughput due to limitation in the sustainable DRAM bandwidth of the FPGA boards. For example, [27] requires to transfer more than 64 byte parallel data from external memory to achieve its peak throughput. On the other hand, the reported architecture from [26] compromises the performance due to bandwidth limitation, and even using 8-bit quantization, [24] is unable to hide all the external communication time under the computation time due to bandwidth limitations. Due to extensive reuse of local data, as discussed in Section III-B2, the proposed CNN accelerator requires to transfer a maximum of 16-byte parallel data to/from the external memory (i.e., DRAM) at its peak throughput for the tested 16-bit precision. Therefore, the maximum required bandwidth of the proposed accelerator is significantly lesser than the sustainable DRAM bandwidth of the FPGA board that has been used for the implementation.

D. Compatibility With State-of-the-Art CNN Models

As discussed in Section IV-A, the proposed architecture natively supports different filter sizes, such as 1×1, 3×3, 5×5, and 7×7. Thus, it can be used for the state-of-the-art CNN models, such as MobileNet-V1, MobileNet-V2, EfficentNet, and many more. Based on our earlier discussion, the proposed CNN-accelerator architecture consists of 16 clusters of 9×6 PE array. Hence, for the state-of-the-art models, some of the clusters can be configured for depth-wise separable convolution (DWC) with larger filter, such as 3×3 or 5×5, while the remaining clusters can be configured for point-wise convolution (PWC) with 1×1 kernels. These clusters can also be configured to work in a pipelined manner to directly perform PWC on the output of DWC to reduce

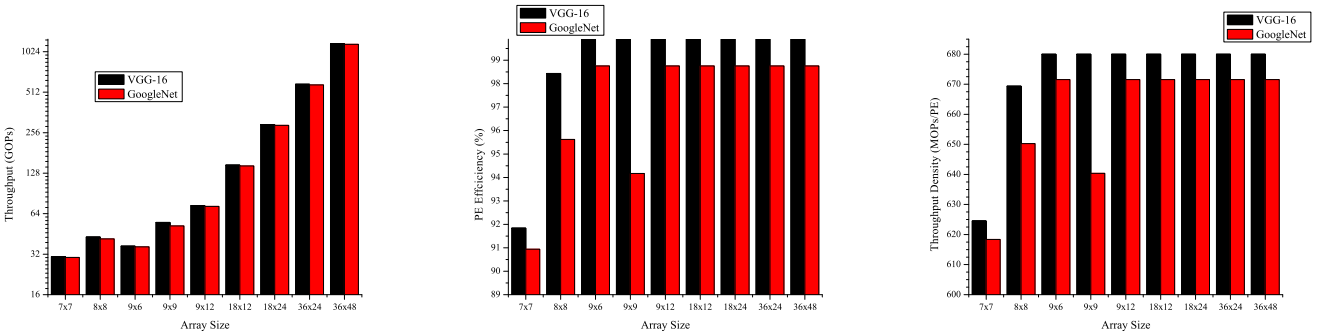

Fig. 5. Various achievable values of throughput (left), PE efficiency (center), and throughput density (right) using different sizes of PE array in the proposed CNN accelerator for VGG-16 and GoogLeNet CNN models.

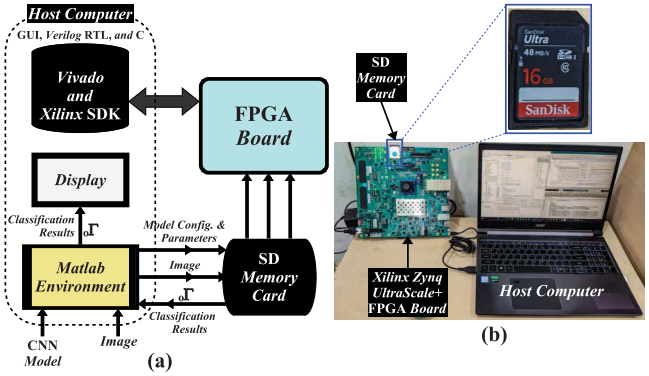

Fig. 6. (a) Schematic representation and (b) real-world snapshot of the test setups for validating the proposed CNN-accelerator prototype.

off-chip data access. Furthermore, the residual connection can be achieved by sending ${}_i\Gamma$ through the ${}_i$Psum port. However, the comparator unit needs to be slightly modified to saturate the data to the value of 6 to achieve ReLU6, as required in MobileNet-V2.

E. Hardware Validation of the Proposed CNN Accelerator

1) Hardware Test Setup and Validation Method: Schematic and real-world views of the test setup for validating the hardware prototype of our CNN accelerator are shown in Fig. 6(a) and (b), respectively. They comprise of three major components: 1) host computer; 2) Zynq-UltraScale+ MPSoC-ZCU102 FPGA board; and 3) external SD memory card. The host computer has been installed with high-end tools, such as MATLAB R2019a, Xilinx Vivado 2018.2, and Xilinx SDK 2018.2. To begin the validation process, a CNN model has been initially imported in the MATLAB environment that is followed by the extraction of model parameters, such as filter weights and biases for different layers of the CNN model. Thereafter, these parameters are converted into 16-bit fixed-point (FP16) format and are stored in the external SD memory card, as multiple binary files. Similarly, pixel data of the input image are also converted to FP16 format and stored in the SD memory card. Consecutively, this SD card is unplugged from the host computer and reconnected to the FPGA board, as shown in Fig. 6. With the aid of Vivado 2018.2 tool, the Verilog hardware-description-language (HDL) code of the proposed CNN accelerator is packed as an AXI-compatible IP. It is then interfaced with the direct memory access (DMA) controller, which is further connected with the AX1-HP port of onboard computer of the FPGA board (for high-speed data transfer from onboard DRAM to CNN accelerator), as illustrated earlier in Fig. 1. It also shows that the configuration and handshake signals between onboard computer and CNN accelerator have been established using AXI-peripheral registers, which are connected with the AXI-*lite* port of the onboard computer. Furthermore, an interrupt signal informs the onboard computer whether the LMCU is ready to receive new data from onboard-DRAM or it requires to offload the ${}_o\Gamma$ data to this DRAM, as shown in Fig. 1.

On the other hand, synthesized hardware information along with bit stream of the CNN accelerator is exported to Xilinx SDK 2018.2 tool, as shown in Fig. 6(a). Here, a stand-alone software application has been written in high-level C-language, which loads the input image and model parameters from the SD memory card to the onboard DRAM of FPGA board. Following that, it sends input image and model parameters of a layer to the LMCU of CNN accelerator using the HP port, as shown in Fig. 1. Simultaneously, it sets the configuration bits according to layer specification. Furthermore, this application software off-loads the ${}_o\Gamma$ data of the layer from LMCU to onboard DRAM and returns them ${}_i\Gamma$ during the processing of subsequent layer. In addition, it stores a part of ${}_o\Gamma$ for each layer and full ${}_o\Gamma$ of the last FC layer in SD memory card. Thereafter, the SD memory card is unplugged from the FPGA board and plugged back to the host computer, to visualize the results in MATLAB environment.

2) Experimental Results: This work presents layer-wise output data for a red, green, and blue (RGB) color image of the dimension $224 \times 224 \times 3$ (this dimension has been specified by the GoogLeNet CNN model) that is processed by the proposed CNN accelerator to classify the object from input image using the GoogLeNet CNN model, as illustrated in Fig. 7. In the first layer, CONV operation has been performed with 64 different filters of dimension $7 \times 7 \times 3$ (three for three different R, G, and B channels of input image) with strides $s = 2$. Such operation enables to search and extract the score for 64 different high-level features in the form of ${}_o\Gamma$, containing 64 feature score matrix of height $\times$ width dimension of 112×112. Here, both these height and width

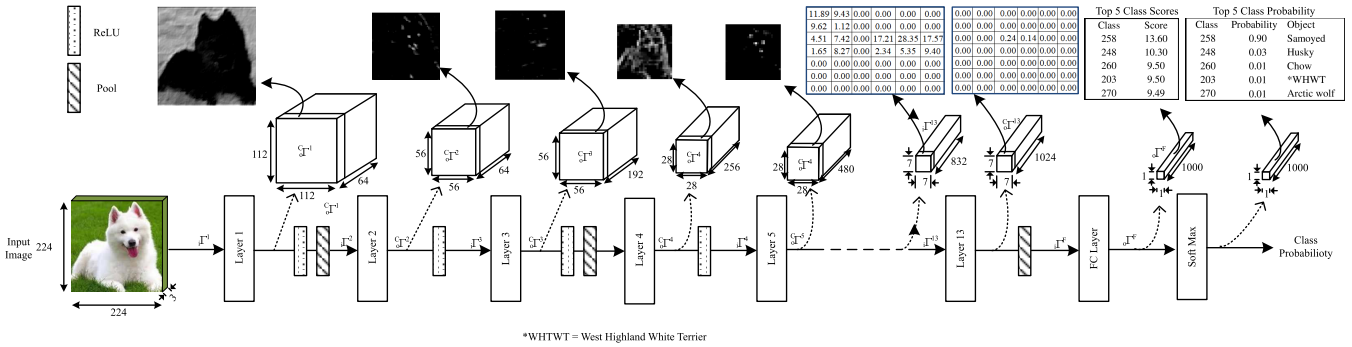

Fig. 7. Schematic representation of layer-wise processing of an image in the proposed CNN-accelerator for the GoogleNet model.

are scaled to (input height)/s and (input width)/s, respectively. Since the $112 \times 112 \times 64$ dimension of ${}_o\Gamma$ is extremely large to visualize here as a matrix, we have plotted one out of these 64 channels as an 112×112 image, mapping each value of the feature scores in ${}_o\Gamma$ as an image pixel. Thereafter, ReLU and max pooling with $s = 2$ have been applied on it that further reduces the dimension to $56 \times 56 \times 64$, which is then fed back to the CNN accelerator for processing layer 2. In this second layer, 64 different filters of dimension $3 \times 3 \times 64$ (64 for 64 different channels of ${}_i\Gamma$) with $s = 1$ are applied that produces ${}_o\Gamma$ of the same dimension, as shown in Fig. 7. To visualize ${}_o\Gamma$, one out of these 64 channels is plotted as an image. Certain features that have been detected produce bright pixel, while dark part denotes the absence or very low score for the searched feature in that region, as shown in Fig. 7. Thereafter, ReLU activation and max pooling with $s = 1$ are applied on ${}_o\Gamma$, before passing it to the next layer. As a result, in those layers where $s > I$ for CONV or pool operation, height and width dimensions of ${}_o\Gamma$ reduce from the same value of ${}_i\Gamma$ by a factor of s. Since after layer 12 onward, the height $\times$ width dimension of ${}_o\Gamma$ is small enough (7×7 or smaller) to be shown as a matrix. Hence, we have presented these values in the tabular format, and the exact numerical value of detection scores for a certain feature is also shown in Fig. 7. Eventually, we have applied the soft-max activation on the output of FC layer, which contains the detection score for all the 1000 classes from the ImageNet dataset to convert these class scores into class probabilities. Hence, the probabilities of Top-5 class have been presented in Fig. 7. All of these Top-5 classes are different classes of dogs or wolf with Top-1 class, showing the correct breed of object in the input image. Thereby, the aforementioned process validates the correct functionality of the proposed CNN-accelerator hardware prototype.

3) Accuracy of the Proposed CNN Accelerator: The suggested uninterrupted processing technique and the corresponding CNN-accelerator architecture are precision-independent. Both of them can be used for any data precision, such as floating point or highly quantized fixed-point representation by simply changing the precision of the computation unit in each PE of our accelerator to the desired precision. Furthermore, throughput of the design also scales up with the reduction in precision. However, for evaluating the suggested architecture, we have designed the data path by considering 16-bit precision. As a result, this work uses both brain float 16 (BF16) as well as 16-bit fixed point (FP16) representations for the data and filter weights. Even though BF16 representation preserves the accuracy of 32-bit floating point representation, it requires higher design efforts to realize the PE unit using the DSP blocks on FPGA and requires extra hardware resources. On the other hand, FP16 representation enables to directly map the PE unit into the inbuilt DSP core blocks of FPGA; however, it alleviates the accuracy of our design by 0.5%.

V. Conclusion

This work focused on solving present-day challenge of achieving higher throughput with better energy efficiency in the design of CNN accelerator. We presented new technique and VLSI architectures for CNN accelerator that delivered better performance than the contemporary implementations in terms of throughput and energy efficiency. To further strengthen our contributions, hardware implementation of our design and its validation with real-world test setup were carried out. Thus, our work demonstrated one of the better ways to design CNN accelerator for the battery operated devices for various artificial intelligence-based edge applications.

References

[1] D. Amodei et al., "Deep speech 2: End-to-end speech recognition in English and Mandarin," *Proc. Int. Conf. Mach. Learn.*, 2016, pp. 173–182.

[2] T. He, W. Huang, Y. Qiao, and J. Yao, "Text-attentional convolutional neural network for scene text detection," *IEEE Trans. Image Process.*, vol. 25, no. 6, pp. 2529–2541, Mar. 2016.

[3] C. Szegedy et al., "Going deeper with convolutions," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2015, pp. 1–9.

[4] H. Li, Z. Lin, X. Shen, J. Brandt, and G. Hua, "A convolutional neural network cascade for face detection," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2015, pp. 5325–5334.

[5] K. Guo et al., "Angel-Eye: A complete design flow for mapping CNN onto embedded FPGA," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 37, no. 1, pp. 35–47, Jan. 2018.

[6] X. Shi et al., "HERO: Accelerating autonomous robotic tasks with FPGA," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, Oct. 2018, pp. 7766–7772.

[7] V. Sze, Y.-H. Chen, T.-J. Yang, and J. S. Emer, "Efficient processing of deep neural networks," *Synth. Lect. Comput. Archit.*, vol. 15, no. 2, pp. 1–341, 2020.

[8] J. Li, K.-F. Un, W.-H. Yu, P.-I. Mak, and R. P. Martins, "An FPGA-based energy-efficient reconfigurable convolutional neural network accelerator for object recognition applications," *IEEE Trans. Circuits Syst. II, Exp. Briefs*, vol. 68, no. 9, pp. 3143–3147, Sep. 2021.

[9] Y.-X. Chen and S.-J. Ruan, "A throughput-optimized channel-oriented processing element array for convolutional neural networks," *IEEE Trans. Circuits Syst. II, Exp. Briefs*, vol. 68, no. 2, pp. 752–756, Feb. 2020.
[10] M. Z. Alom et al., "A state-of-the-art survey on deep learning theory and architectures," *Electron*, vol. 8, no. 3, p. 292, Mar. 2019.
[11] K. Siu, D. M. Stuart, M. Mahmoud, and A. Moshovos, "Memory requirements for convolutional neural network hardware accelerators," in *Proc. IEEE Int. Symp. Workload Characterization (IISWC)*, Sep. 2018, pp. 111–121.
[12] A. A. Gilan, M. Emad, and B. Alizadeh, "FPGA-based implementation of a real-time object recognition system using convolutional neural network," *IEEE Trans. Circuits Syst. II, Exp. Briefs*, vol. 67, no. 4, pp. 755–759, Apr. 2019.
[13] S. S. L. Oskouei, H. Golestani, M. Hashemi, and S. Ghiasi, "CNNdroid: GPU-accelerated execution of trained deep convolutional neural networks on android," in *Proc. 24th ACM Int. Conf. Multimedia*, Oct. 2016, pp. 1201–1205.
[14] N. P. Jouppi et al., "In-datacenter performance analysis of a tensor processing unit," in *Proc. 44th Annu. Int. Symp. Comput. Archit.*, 2017, pp. 1–12.
[15] S. Moini, B. Alizadeh, M. Emad, and R. Ebrahimpour, "A resource-limited hardware accelerator for convolutional neural networks in embedded vision applications," *IEEE Trans. Circuits Syst. II, Exp. Briefs*, vol. 64, no. 10, pp. 1217–1221, Oct. 2017.
[16] J. Qiu et al., "Going deeper with embedded FPGA platform for convolutional neural network," in *Proc. ACM/SIGDA Int. Symp. Field-Program. Gate Arrays*, Feb. 2016, pp. 25–35.
[17] Y. H. Chen, T. Krishna, J. S. Emer, and V. Sze, "Eyeriss: An energy-efficient reconfigurable accelerator for deep convolutional neural networks," *IEEE J. Solid-State Circuits*, vol. 52, no. 1, pp. 127–138, Jan. 2016.
[18] H. Jia, H. Valavi, Y. Tang, J. Zhang, and N. Verma, "A programmable heterogeneous microprocessor based on bit-scalable in-memory computing," *IEEE J. Solid-State Circuits*, vol. 55, no. 9, pp. 2609–2621, Sep. 2020.
[19] D. T. Nguyen, T. N. Nguyen, H. Kim, and H. J. Lee, "A high-throughput and power-efficient FPGA implementation of YOLO CNN for object detection," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 27, no. 8, pp. 1861–1873, Aug. 2019.
[20] D. T. Nguyen, H. Kim, and H.-J. Lee, "Layer-specific optimization for mixed data flow with mixed precision in FPGA design for CNN-based object detectors," *IEEE Trans. Circuits Syst. Video Technol.*, vol. 31, no. 6, pp. 2450–2464, Jun. 2020.
[21] P. Darbani, N. Rohbani, H. Beitollahi, and P. Lotfi-Kamran, "RASHT: A partially reconfigurable architecture for efficient implementation of CNNs," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 30, no. 7, pp. 860–868, Jul. 2022, doi: 10.1109/TVLSI.2022.3167449.
[22] L. Cavigelli, D. Gschwend, C. Mayer, S. Willi, B. Muheim, and L. Benini, "Origami: A convolutional network accelerator," in *Proc. 25th Ed., Great Lakes Symp. VLSI*, May 2015, pp. 199–204.
[23] Z. Du et al., "ShiDianNao: Shifting vision processing closer to the sensor," in *Proc. 42nd Annu. Int. Symp. Comput. Archit.*, Jun. 2015, pp. 92–104.
[24] C. Wu, J. Zhuang, K. Wang, and L. He, "MP-OPU: A mixed precision FPGA-based overlay processor for convolutional neural networks," in *Proc. Int. Conf. Field Program. Log. Appl.*, 2021, pp. 33–37.
[25] J. Zhang et al., "A low-latency FPGA implementation for real-time object detection," in *Proc. IEEE Int. Symp. Circuits Syst.*, May 2021, pp. 1–5.
[26] F. Spagnolo, S. Perri, F. Frustaci, and P. Corsonello, "Reconfigurable convolution architecture for heterogeneous systems on-chip," in *Proc. 9th Medit. Conf. Embedded Comput. (MECO)*, Jun. 2020, pp. 1–5.
[27] D. Wang, K. Xu, J. Guo, and S. Ghiasi, "DSP-efficient hardware acceleration of convolutional neural network inference on FPGAs," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 39, no. 12, pp. 4867–4880, Dec. 2020.

Md Najrul Islam (Graduate Student Member, IEEE) received the Bachelor of Engineering degree in electronics and telecommunication engineering from Tripura University, Agartala, India, in 2016, and the M.Tech. degree in VLSI from the National Institute of Technology Meghalaya, Shillong, India, in 2018. He is currently pursuing the Ph.D. degree with the School of Computing and Electrical Engineering, Indian Institute of Technology Mandi, Mandi, India.

His current research focuses on developing VLSI architectures of energy and area efficient reconfigurable deep-neural-network hardware accelerators for edge applications.

Rahul Shrestha (Senior Member, IEEE) received the Bachelor of Engineering degree in telecommunication engineering from the B.M.S. College of Engineering, Bengaluru, India, in 2008, and the Ph.D. degree in electronics and electrical engineering from the Indian Institute of Technology Guwahati, Guwahati, India, in 2014.

Currently, he holds the position of an Associate Professor at the School of Computing and Electrical Engineering, Indian Institute of Technology Mandi, Mandi, India. His primary research interests are digital VLSI architectures and its transformation into ASIC-chip or FPGA-prototype for the real-world applications of signal processing, wireless communication, deep neural networks, forward-error-correction channel decoders, cognitive radio, and cooperative spectrum sensing.

Shubhajit Roy Chowdhury (Senior Member, IEEE) received the Ph.D. degree from the Department of Electronics and Telecommunication Engineering, Jadavpur University, Kolkata, India, in 2010.

He is currently an Associate Professor at the School of Computing and Electrical Engineering, Indian Institute of Technology Mandi, Mandi, India. Previously, he served as an Assistant Professor at the Center for VLSI and Embedded Systems Technology, IIIT Hyderabad, Hyderabad, India.

Dr. Roy Chowdhury was a recipient of the University Gold Medals for his B.E. and M.E. degrees in 2004 and 2006, respectively, the Altera Embedded Processor Designer Award in 2007, and the winner of five best paper awards. He received the Award of the Fellow of the Society of Applied Biotechnology (FSAB) by the Society of Applied Biotechnology in 2012. He is also awarded Young Engineers' Award 2012–2013 by the Institution of Engineers, India, for his outstanding contribution in the field of electronics and telecommunication engineering.